Verilog RTL Design & Verification Test

Instructions for students

This test has 30 questions: 10 easy (Section A), 10 moderate (Section B) and 10 difficult (Section C). Each question covers either a combinational or a sequential circuit.

Audience and level: second-year Computer Science students in a Digital Design course. Only basic Verilog is needed: `assign`, `always @(*)`, `always @(posedge clk)`, `if`/`case`, module instantiation, parameters and simple FSMs. No timing analysis, clock-domain crossing or circuit-level electronics is expected.

For **every** question you must submit:

1. The synthesizable Verilog design module (RTL, not behavioral delays).
2. A self-checking testbench that applies stimulus, compares outputs against expected values (use `$display` with PASS/FAIL, or `$error`) and ends with `$finish`.
3. A short note on the simulation result (waveform screenshot or console log showing all tests passed).

Section	Questions	Marks each	Total
A: Easy	Q1–Q10	2	20
B: Moderate	Q11–Q20	4	40
C: Difficult	Q21–Q30	6	60

Marks split per question: 50% design, 40% testbench quality (coverage of corner cases, self-checking), 10% result evidence. Attempt the number of questions your instructor specifies (suggested: all of A, any 7 of B, any 5 of C, so students can choose within the time available).

General rules: use `timescale 1ns/1ps`; resets must be stated as synchronous or asynchronous in your code comments; no `initial` blocks or `#delays` inside the design module; testbench file name must be `tb_<module>.v`.

Section A: (Q1–Q10, 2 marks each)

Five combinational (Q1–Q5) and five sequential (Q6–Q10) circuits. For every question: write the RTL module, write a self-checking testbench, simulate and show the result.

Q1 (Combinational): Basic logic gates. Design a module `gates` with inputs `a`, `b` and outputs `y_and`, `y_or`, `y_nand`, `y_nor`, `y_xor`, `y_xnor`. Testbench: apply all four input combinations and compare each output with the expected truth table.

Q2 (Combinational): Full adder from half adders. Design a `half_adder` module, then build a `full_adder` (inputs `a`, `b`, `cin`; outputs `sum`, `cout`) structurally using two half adders and an OR gate. Testbench: apply all 8 input combinations and check against `{cout,sum} = a + b + cin`.

Q3 (Combinational): 4-bit 2-to-1 multiplexer. Design a 2-to-1 mux with 4-bit data inputs `d0`, `d1`, select `s` and output `y`, using the conditional operator. Testbench: test both select values with several data patterns, including all-zeros and all-ones.

Q4 (Combinational): 4-bit magnitude comparator. Design a comparator with 4-bit inputs `a`, `b` and outputs `gt`, `eq`, `lt`. Testbench: exhaustively test all 256 input pairs and check that exactly one output is high each time.

Q5 (Combinational): 3-to-8 decoder with enable. Design a decoder with 3-bit input `in`, enable `en` and 8-bit one-hot output `out` (all zeros when `en = 0`). Testbench: loop over all inputs with `en` high and low, and check the output against `(1 << in)`.

Q6 (Sequential): D flip-flop with asynchronous reset. Design a positive-edge D flip-flop with active-low asynchronous reset `rst_n`. Testbench: generate a 10 ns clock, apply data changes, assert reset in the middle of a clock cycle, and verify that `q` clears immediately without waiting for the clock edge.

Q7 (Sequential): 4-bit up counter. Design a counter with synchronous active-high reset and enable `en`. It counts 0 to 15 and wraps to 0. Testbench: check counting, wrap-around, hold when `en = 0`, and reset behaviour, using a reference counter variable inside the testbench.

Q8 (Sequential): T flip-flop. Design a T flip-flop with synchronous reset that toggles `q` on each clock edge when `t = 1` and holds when `t = 0`. Testbench: apply `t` patterns including back-to-back toggles, and compare `q` with an expected value computed in the testbench each cycle.

Q9 (Sequential): 8-bit register with load enable. Design an 8-bit register with synchronous reset, load enable `load`, data input `d` and output `q`. Testbench: load several random values, check that `q` holds when `load = 0`, and check reset.

Q10 (Sequential): 4-bit serial-in parallel-out shift register. Design a shift register that shifts in serial input `sin` at every positive clock edge, with the newest bit entering at the LSB and a 4-bit parallel output `q`. Testbench: shift in the sequence 1011 and verify `q` after every clock edge, then also test the reset.

Section B: (Q11–Q20, 4 marks each)

Five combinational (Q11–Q15) and five sequential (Q16–Q20) circuits. Testbenches should use loops and compare against an expected value computed in the testbench.

Q11 (Combinational): 4-bit ripple-carry adder. Reuse your `full_adder` from Q2 and instantiate it four times to build a 4-bit adder. Inputs `a[3:0]`, `b[3:0]`, `cin`; outputs `sum[3:0]`, `cout`. Testbench: use nested loops to test all 512 combinations, compare with `a + b + cin`, and print the number of errors at the end.

Q12 (Combinational): 4-to-2 priority encoder. Design an encoder with 4-bit request input `req` (bit 3 has the highest priority), 2-bit output `code` and a `valid` output that is low when no request is active.

Testbench: test all 16 input values and compare with the expected code written down in a table in the testbench.

Q13 (Combinational): 4-bit mini ALU. Design an ALU with 4-bit inputs `a`, `b`, a 2-bit `op` and 4-bit output `y`, plus a `zero` flag. Operations: 00 add, 01 subtract, 10 AND, 11 OR. Testbench: test every operation with at least 10 different input pairs, including 0 and 15, and check `y` and `zero`.

Q14 (Combinational): BCD to seven-segment decoder. Design a decoder with a 4-bit BCD input and a 7-bit output `{a,b,c,d,e,f,g}` for a common-cathode display (1 = segment on). Inputs 10 to 15 must turn all segments off. Testbench: check all 16 inputs against a lookup table in the testbench.

Q15 (Combinational): 8-bit parity generator and checker. Design a module that takes an 8-bit data input and outputs an even-parity bit, and a second module that takes 9 bits (data plus parity) and outputs `error = 1` if the parity is wrong. Testbench: send all 256 data values through the generator, check the checker reports no error, then flip one bit and check it reports an error.

Q16 (Sequential): 4-bit up/down counter with load. Design a counter with synchronous reset, synchronous `load` (with data `d`), direction input `up` and enable `en`. It wraps from 15 to 0 when counting up and from 0 to 15 when counting down. Testbench: check counting in both directions, wrap-around, load, enable and reset, comparing with a variable that tracks the expected value.

Q17 (Sequential): 4-bit shift register with modes. Design a register with a 2-bit `mode` input: 00 hold, 01 shift left (serial input `sin` enters at the LSB), 10 shift right (serial input `sin` enters at the MSB), 11 parallel load. Include a synchronous reset. Testbench: test every mode and also switch modes on consecutive clock edges, checking the output after each edge.

Q18 (Sequential): "101" sequence detector (Mealy FSM). Design a Mealy FSM with serial input `x` and output `z` that goes high when the last 1 of the pattern 101 arrives. Overlapping is allowed (10101 gives two detections). Draw the state diagram and the state table as part of your answer. Testbench: send the stream 1 0 1 0 1 1 0 1 0 0 1 0 1 and check `z` after every clock against a list of expected values.

Q19 (Sequential): Single traffic light controller (Moore FSM). Design a controller for one traffic light with outputs `red`, `yellow`, `green`. Sequence: green for 5 clock cycles, yellow for 2, red for 5, then repeat. Use a state register and a small counter. Draw the state diagram. Testbench: run for at least 40 clock cycles, check that exactly one light is on at any time, and check how many cycles each light stays on.

Q20 (Sequential): Two-digit BCD counter (00 to 99). First design a 4-bit BCD counter that counts 0 to 9 and then returns to 0, with an output `carry` that is high when the count is 9. Then connect two of them so that the second counter increments when the first one rolls over. Testbench: let the circuit run for 250 clock cycles and check that it counts 00 to 99, wraps to 00, and that reset works.

Section C: (Q21–Q30, 6 marks each)

Five combinational (Q21–Q25) and five sequential (Q26–Q30) designs. These are longer designs that combine several modules or an FSM with a datapath. Testbenches must check results automatically (print PASS/FAIL), use loops or `$random` for many test values, and finish with a summary line giving the number of errors.

Q21 (Combinational): 8-bit ALU with flags. Design an ALU with 8-bit inputs `a`, `b`, a 3-bit `op` and 8-bit output `y`. Operations: 000 add, 001 subtract, 010 AND, 011 OR, 100 XOR, 101 NOT `a`, 110 shift `a` left by 1, 111 shift `a` right by 1. Flags: `zero`, `carry` (for add and subtract) and `negative` (MSB of `y`). Testbench: test every operation with directed cases (0, 255, carry and borrow cases) and at least 100 random pairs, comparing with a reference `case` statement in the testbench.

Q22 (Combinational): 4×4 unsigned multiplier. Build a multiplier from AND gates for the partial products and full adders (reuse your `full_adder`) to add them. Inputs `a[3:0]`, `b[3:0]`; output `p[7:0]`. Do not use the `*` operator in the design. Testbench: test all 256 input pairs and compare with `a * b`.

Q23 (Combinational): 8-bit barrel shifter. Design a shifter with a 3-bit shift amount and a 1-bit direction input (left or right, logical shift). Build it from three stages of 2-to-1 muxes (shift by 1, by 2, by 4) instead of the `<<` and `>>` operators. Testbench: test every shift amount in both directions with at least 5 different data values and compare with the `<<` and `>>` operators.

Q24 (Combinational): BCD adder. Design a circuit that adds two BCD digits (0 to 9) plus a carry-in and gives a BCD sum digit and a carry-out. Use a 4-bit binary adder followed by a correction step (add 6 when the binary result is more than 9). Testbench: test all valid digit pairs (100 × 2 carry-in values) and check the result against the decimal value `a + b + cin`.

Q25 (Combinational): Maximum of four numbers. Design a module with four 8-bit unsigned inputs and outputs the largest value and the index (0 to 3) of the input holding it. Build it from comparators and muxes (compare in pairs, then compare the two winners). If values tie, the lower index wins. Testbench: test at least 200 random sets, plus the cases where all values are equal and where the maximum is at each of the four positions.

Q26 (Sequential): Synchronous FIFO. Design a FIFO with 8-bit data and 8 entries, inputs `wr_en`, `rd_en`, `din` and outputs `dout`, `full`, `empty`. Use a memory array and read/write pointers. Writes when full and reads when empty must be ignored. Testbench: write 8 values and check that `full` goes high, try one more write, then read all values back and check they come out in the same order and that `empty` goes high. Also test a few cycles of simultaneous read and write.

Q27 (Sequential): Vending machine controller (FSM). A machine accepts 5-rupee and 10-rupee coins (inputs `coin5`, `coin10`, never active together) and dispenses the item (`dispense = 1` for one cycle) when the total reaches 15 rupees. If 20 rupees is reached (10 + 10), it dispenses and also gives 5 rupees change (`change = 1`). After dispensing it returns to the start. Draw the state diagram, then write the FSM. Testbench: test the coin sequences 5-5-5, 5-10, 10-5, 10-10, 5-5-10 and reset in the middle of a sequence.

Q28 (Sequential): 4-bit shift-and-add multiplier. Design a multiplier with inputs `start`, `a[3:0]`, `b[3:0]` and outputs `product[7:0]` and `done`. A control FSM should repeat four times: if the lowest bit of the multiplier is 1, add the multiplicand to the product, then shift. `done` goes high when the result is ready. Testbench: test all 256 input pairs (wait for `done` each time) and compare with `a * b`.

Q29 (Sequential): 4-requester round-robin arbiter. Design an arbiter with inputs `req[3:0]` and output `grant[3:0]` (one-hot or all zero). After a requester is served, the next requester in order gets the highest priority, so nobody waits forever. Testbench: check that `grant` is never given to a requester that

is not requesting, that at most one grant is active, and that with all four requests held high the grants go 0, 1, 2, 3, 0, 1, ... in order.

Q30 (Sequential): Stack (LIFO memory). Design a stack with 8-bit data and 8 entries, inputs `push`, `pop`, `din` and outputs `dout` (the top value), `full`, `empty`. Use a memory array and a stack pointer. Pushes when full and pops when empty must be ignored. Testbench: push 8 values and check `full`, try one more push, pop all values and check they come out in reverse order and that `empty` goes high. Also test pop on an empty stack and push/pop on alternate cycles.